

MÓDULO AVANZADO — MCP con PostgreSQL

Codex + Postgres MCP Server

Schema Discovery · Query Generation · EXPLAIN · Migration · Security
Plan de Formación — OpenAI Codex para Desarrolladores Nivel: Avanzado · Duración
estimada: 2–3 horas (teoría + 3 laboratorios) Versión 1.0 — Febrero 2026

1. Introducción

Codex puede conectarse a bases de datos PostgreSQL mediante MCP (Model Context Protocol), un estándar abierto que permite al agente acceder a herramientas externas de forma controlada. Con un servidor MCP de PostgreSQL, Codex puede inspeccionar esquemas, generar queries correctas basadas en el schema real, analizar planes de ejecución (EXPLAIN) y apoyar migraciones incrementales, todo sin salir del CLI o IDE. Este módulo enseña a configurar la conexión MCP, aplicar guardrails de seguridad (read-only, usuario dedicado, credenciales por variables de entorno) y ejecutar tareas reales de desarrollo sobre PostgreSQL asistidas por Codex.

2. Objetivos de aprendizaje

#	Objetivo	Evidencia de logro
O1	Explicar MCP (host/cliente/servidor, transporte stdio/http) y cómo lo usa Codex.	Diagrama mental correcto del flujo Codex → MCP → PostgreSQL.
O2	Configurar un servidor MCP de PostgreSQL conectado a Codex CLI/IDE de forma reproducible.	codex mcp list muestra el servidor activo.
O3	Consultar schema y ejecutar queries con guardrails (read-only por defecto).	3 queries correctas generadas y verificadas.
O4	Aplicar seguridad operativa: mínimo privilegio, credenciales aisladas, MCP allowlist.	Configuración segura verificable en config.toml.

3. Contenidos teóricos

3.1 MCP en Codex: qué es y cómo se configura

MCP conecta al agente Codex con herramientas y recursos externos (bases de datos, APIs, documentación) a través de servidores MCP. Codex soporta dos tipos de transporte:

Transporte	Descripción	Ejemplo
STDIO	Proceso local lanzado por un comando. Codex lo arranca y se comunica via stdin/stdout.	npx @modelcontextprotocol/server-postgres ...
Streamable HTTP	Servidor remoto accesible por URL. Codex hace peticiones HTTP.	https://developers.openai.com/mcp

La configuración se almacena en config.toml, compartida entre CLI e IDE. No es necesario configurar dos veces.

Añadir servidor MCP via CLI

```
codex mcp add postgres_lab \  
-- npx -y @modelcontextprotocol/server-postgres \  
"postgres://codex_ro:readonly2026@localhost:5432/labdb"
```

Verificar servidores configurados

codex mcp list

Configuración directa en config.toml

~/.codex/config.toml (global) o .codex/config.toml (proyecto trusted)

[mcp_servers.postgres_lab]

command = "npx"

args = ["-y", "@modelcontextprotocol/server-postgres", "postgresql://codex_ro:readonly2026@localhost:5432/labdb"]

startup_timeout_sec = 15.0 # default 10s, subir si npx descarga lento

 **Seguridad de credenciales:** La connection string va como argumento del proceso. En entorno corporativo, considerar un wrapper script que lea la URL de un vault o variable de entorno y la pase como argumento, evitando hardcodear credenciales en ficheros versionados. Este servidor es **read-only por diseño**: todas las queries se ejecutan dentro de una transacción READ ONLY.

Opciones avanzadas de MCP servers

Parámetro	Tipo	Descripción
enabled	bool	Activar/desactivar sin borrar config (default true).
required	bool	Si true, Codex falla al arrancar si el servidor no conecta.
env	{ key = val }	Variables de entorno copiadas al proceso MCP.
env_vars	[string]	Variables forwarded desde el entorno padre.
enabled_tools	[string]	Allowlist: solo estas tools están disponibles.
disabled_tools	[string]	Denylist: ocultar tools específicas.
startup_timeout_sec	float	Timeout de arranque (default 10s).
tool_timeout_sec	float	Timeout por invocación de tool (default 60s).

3.2 MCP + PostgreSQL: herramientas disponibles

El Reference PostgreSQL MCP Server (@modelcontextprotocol/server-postgres) expone capacidades a través de MCP tools y MCP resources:

MCP Tool:

Tool	Descripción	Modo
MCP		
query	Ejecuta queries SQL. Todas se ejecutan dentro de una transacción READ ONLY.	Read-only (forzado)

MCP Resources (schema automático):

Resource URI	Descripción
postgres://<host>/<table>/schema	JSON schema de cada tabla: columnas, tipos de datos. Descubierto automáticamente desde los metadatos de la base de datos.

 **Nota:** Este servidor es **read-only por diseño**: no hay modo write. Todas las queries van en transacción READ ONLY. Para migraciones DDL (Lab L-PG-3), las sentencias se ejecutan directamente con psql, no a través del MCP.

3.3 Selección de servidor MCP (criterios corporativos)

En entorno corporativo, el criterio de selección no es “la primera implementación que funcione”:

- Mantenimiento y actividad del repositorio (commits recientes, releases con tag).
- Modo read-only vs read/write configurable (principio de mínimo privilegio).
- Soporte de transporte: STDIO (local) y/o HTTP (remoto).
- Observabilidad: logs de queries ejecutadas, métricas.
- Facilidad para aislar secretos (wrapper script, vault, o env_vars si el servidor lo soporta).

• Pin de versión: fijar @modelcontextprotocol/server-postgres en vez de latest.				
Servidor MCP	Enfoque	Read-only	EXPLAIN	Transporte
Reference PostgreSQL MCP Schema + (modelcontextprotocol)	queries read-only. Usado en este módulo.	Sí (forzado por transacción READ ONLY)	Via query: SELECT * FROM EXPLAIN...	STDIO
Postgres MCP Pro (crystaldba)	Dev + tuning + índices hipotéticos	Configurable (restricted mode)	Sí (índices hipotéticos)	STDIO
Supabase Postgres MCP	Gestión plataforma Supabase	Read/Write (features Supabase)	No	STDIO

⚠ **Riesgo de supply chain MCP:** Existe evidencia pública de incidentes con servidores MCP maliciosos en ecosistemas de paquetes. En entorno corporativo: allowlist en requirements.toml, pin de versiones, y revisión de código del servidor antes de adoptar.

3.4 Seguridad y guardrails

3.4.1 Usuario dedicado read-only en PostgreSQL

-- Crear usuario con permisos mínimos

```
CREATE ROLE codex_ro WITH LOGIN PASSWORD 'lab_readonly_2026';
GRANT CONNECT ON DATABASE labdb TO codex_ro;
GRANT USAGE ON SCHEMA public TO codex_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO codex_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO codex_ro;
```

3.4.2 Entorno aislado

- NUNCA conectar a producción directamente. Usar réplica, snapshot o Docker con datos de ejemplo.
- La connection string va como argumento del proceso MCP en config.toml. En entorno corporativo, usar wrapper script que lea de vault.
- Nunca pegar connection strings en prompts de Codex (viajan a la API y quedan en contexto).

3.4.3 MCP allowlist en requirements.toml (Enterprise)

requirements.toml (admin-enforced, no overridable)

```
[[mcp_servers]]
```

```
id = "postgres_lab"
```

```
identity = { command = "npx" }
```

Si mcp_servers está presente pero vacío en requirements.toml, Codex desactiva TODOS los servidores MCP. Tanto el id como la identity (command o url) deben coincidir para que el servidor sea permitido.

3.4.4 Control de red y approvals

El servidor MCP STDIO se comunica via stdin/stdout con Codex, no necesita red abierta para el protocolo MCP en sí. La conexión a PostgreSQL la hace el proceso MCP directamente (fuera del sandbox de Codex). Mantener network_access = false en el sandbox de Codex y approval_policy = on-request.

3.4.5 Control de tools con enabled_tools

El Reference PostgreSQL MCP Server solo tiene la tool "query"

y ya es read-only por diseño. Para desactivar queries y dejar

solo el schema discovery via resources:

```
[mcp_servers.postgres_lab]
```

```
command = "npx"
args = ["-y", "@modelcontextprotocol/server-postgres", "postgresql://codex_ro:readonly2026@localhost:5432/labdb"]
disabled_tools = ["query"] # solo schema via MCP resources
```

4. Buenas prácticas

4.1 Schema-first / Read-first

Siempre pedir primero “mapea el schema” antes de generar queries. Esto ancla el contexto de Codex en el schema real (via los MCP resources postgres://<host>/<table>/schema y la tool query para consultas de catálogo) y elimina alucinaciones de tablas o columnas inexistentes.

4.2 Solo SELECT por defecto

El servidor MCP en modo restricted solo permite SELECT. Habilitar write solo en laboratorio controlado con rol dedicado y approval explícito. En AGENTS.md: “Solo genera queries SELECT. Para DDL/DML, usa la plantilla migration-step y requiere aprobación explícita.”

4.3 Trazabilidad de queries

Cada query generada por Codex debe guardarse en fichero .sql en el repo del laboratorio, con comentario SQL indicando propósito y resultado esperado. Esto permite auditoría y reproducción.

4.4 EXPLAIN antes de optimizar

No pedir “optimiza la base de datos” sin baseline. Primero: pedir a Codex que ejecute EXPLAIN ANALYZE via la tool query del MCP en queries lentas, revisar índices existentes y cardinalidades reales. Ejemplo: “Ejecuta EXPLAIN ANALYZE de esta query via postgres_lab y analiza el plan.”

4.5 Pin de versión del servidor MCP

En CI y equipos, pinear la versión exacta del paquete: @modelcontextprotocol/server-postgres. Evitar latest para prevenir roturas por actualizaciones no probadas y riesgos de supply chain.

5. Errores comunes

5.1 Permisos de escritura por comodidad

Problema: Usar un superuser para el servidor MCP “porque es más fácil”. **Solución:** Usuario read-only dedicado (codex_ro). Write solo con rol limitado en laboratorio.

5.2 Conectar a producción

Problema: Usar la connection string de prod para “probar rápido” con Codex. **Solución:** Docker local con dataset de ejemplo o réplica read-only dedicada.

5.3 Servidor MCP sin verificar

Problema: Instalar un servidor MCP desde un paquete npm desconocido sin revisar código ni pinear versión. **Solución:** Allowlist en requirements.toml + pin de versión + revisión del repositorio.

5.4 Optimizar sin baseline

Problema: Pedir “optimiza la base de datos” sin EXPLAIN ni métricas de partida. **Solución:** Siempre EXPLAIN ANALYZE via la tool query primero. Luego pedir mejoras específicas con datos.

5.5 Credenciales en prompts

Problema: Pegar la connection string directamente en un prompt de Codex. **Solución:** La connection string va como argumento en config.toml. En entorno corporativo, usar un wrapper script que lea de vault. Nunca incluir credenciales en el texto del prompt.

6. Casos de uso reales en desarrollo

6.1 Onboarding en modelo de datos

Nuevo developer se incorpora al equipo: “Explícame el modelo de datos de este proyecto”. Codex usa los MCP resources de schema y la tool query para consultar information_schema, generando docs/schema-map.md con tablas, relaciones, tipos de datos y cardinalidades. El developer entiende el modelo en minutos en vez de horas.

6.2 Bugfix de query incorrecta

Query con JOIN mal planteado que duplica filas en un informe de ventas. Codex ejecuta la query buggy via la tool query del MCP, identifica el cross join implícito, propone fix con JOIN explícito y genera test de regresión con pytest que verifica invariantes (conteos, totales).

6.3 Migración incremental

Añadir columna nullable con backfill por lotes, manteniendo backward compatibility. Scripts up/down en sql/migrations/, feature flag para activar la nueva funcionalidad, y verificación con tests antes y después de la migración.

6.4 Performance con EXPLAIN

Query lenta en dashboard de métricas. Codex usa la tool query para ejecutar EXPLAIN ANALYZE y obtener el plan de ejecución, identifica un sequential scan en una tabla grande y sugiere índice específico. La verificación del índice se hace creándolo en el lab y repitiendo el EXPLAIN.

7. Laboratorios prácticos

 **Prerequisitos comunes:** Docker + PostgreSQL 16. Si la organización no permite Docker, sustituir por PostgreSQL local o contenedor corporativo equivalente. Codex CLI instalado con soporte MCP disponible. Node.js (para npx).

7.1 Lab L-PG-1 — Conectar Codex a Postgres MCP y explorar schema

 **Objetivo:** Levantar Postgres de laboratorio, conectar servidor MCP de PostgreSQL a Codex, pedir inventario del schema y 3 queries útiles y verificables.

Paso 1: Setup del proyecto

```
mkdir /tmp/codex-lab-mcp-pg && cd /tmp/codex-lab-mcp-pg
git init
mkdir -p sql docs .codex
```

Paso 2: Levantar PostgreSQL con Docker

```
# docker-compose.yml
```

```
services:
  postgres:
    image: postgres:16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labadmin
      POSTGRES_PASSWORD: labpass2026
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
```

```
volumes:
```

```
  pgdata:
```

Paso 3: Dataset de ejemplo + usuario read-only

```
-- init.sql
```

```
-- Schema: tienda online simplificada
```

```
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  price NUMERIC(10,2) NOT NULL CHECK (price > 0),
  stock INTEGER DEFAULT 0
);
```

```
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  customer_id INTEGER REFERENCES customers(id),
  total NUMERIC(10,2) NOT NULL,
  status VARCHAR(20) DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE TABLE order_items (
  id SERIAL PRIMARY KEY,
  order_id INTEGER REFERENCES orders(id),
  product_id INTEGER REFERENCES products(id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(10,2) NOT NULL
);
```

-- Datos de ejemplo

INSERT INTO customers (name, email) **VALUES**

('Ana Garcia', 'ana@example.com'),
('Carlos Lopez', 'carlos@example.com'),
('Maria Fernandez', 'maria@example.com');

INSERT INTO products (name, price, stock) **VALUES**

('Laptop Pro', 1299.99, 50),
('Teclado Mecanico', 89.99, 200),
('Monitor 27"', 449.99, 75);

INSERT INTO orders (customer_id, total, status) **VALUES**

(1, 1389.98, 'completed'),
(2, 89.99, 'pending'),
(3, 449.99, 'shipped');

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES

(1, 1, 1, 1299.99), (1, 2, 1, 89.99),
(2, 2, 1, 89.99),
(3, 3, 1, 449.99);

-- Usuario read-only para MCP

CREATE ROLE codex_ro **WITH LOGIN PASSWORD** 'readonly2026';

GRANT CONNECT ON DATABASE labdb **TO** codex_ro;

GRANT USAGE ON SCHEMA public **TO** codex_ro;

GRANT SELECT ON ALL TABLES IN SCHEMA public **TO** codex_ro;

ALTER DEFAULT PRIVILEGES IN SCHEMA public

GRANT SELECT ON TABLES TO codex_ro;

docker compose up -d

sleep 3

docker compose exec postgres psql -U labadmin -d labdb -c '\dt'

Paso 4: Configurar servidor MCP en Codex

Opción A: via CLI

codex mcp add postgres_lab \

-- npx -y @modelcontextprotocol/server-postgres \

"postgresql://codex_ro:readonly2026@localhost:5432/labdb"

Verificar

codex mcp list

Opción B: en .codex/config.toml del proyecto

[mcp_servers.postgres_lab]

command = "npx"

args = ["-y", "@modelcontextprotocol/server-postgres", "postgresql://codex_ro:readonly2026@localhost:5432/labdb"]

startup_timeout_sec = 15.0

💡 **Nota:** Si npx tarda en descargar el paquete la primera vez, ejecutar npx -y @modelcontextprotocol/server-postgres --help antes de lanzar Codex para que quede en caché. Si el handshake falla, subir startup_timeout_sec a 30.

Paso 5: Explorar schema con Codex

codex

Turno 1: Schema discovery

Usa las herramientas MCP de postgres_lab para:

1. Consultar los resources de schema disponibles
2. Listar tablas, relaciones y columnas clave
3. Genera docs/schema-map.md con el mapa completo

Turno 2: Queries

Propone 3 queries SELECT útiles para esta tienda

y ejecútalas via la tool query de postgres_lab:

1. Top 3 clientes por total gastado

2. Productos sin ventas

3. Resumen de pedidos por estado

Guarda en sql/q1.sql, sql/q2.sql, sql/q3.sql

Ejecuta cada una y muestra resultados.

Verificación L-PG-1

Verificación	Resultado esperado
codex mcp list	postgres_lab activo
docs/schema-map.md	4 tablas, columnas, PKs, FKs, tipos de datos
sql/q1.sql	Query correcta: top clientes por gasto total
sql/q2.sql	Query correcta: productos sin ventas
sql/q3.sql	Query correcta: resumen pedidos por estado
Resultados ejecutados	Conteos coherentes con datos insertados

Limpieza L-PG-1 (obligatoria)

`cd /tmp && rm -rf codex-lab-mcp-pg`

`docker compose -f /tmp/codex-lab-mcp-pg/docker-compose.yml down -v 2>/dev/null`

Si se agregó MCP global: editar ~/.codex/config.toml

y borrar [mcp_servers.postgres_lab]

7.2 Lab L-PG-2 — Query clinic: del bug al test de regresión

 **Objetivo:** Partir de una query errónea (cross join implícito), llegar a: diagnóstico + fix + test automatizado con pytest.

Paso 1: Setup (igual que L-PG-1)

Levantar Postgres con Docker + dataset de ejemplo + usuario codex_ro + MCP configurado (repetir pasos 1–4 de L-PG-1).

Paso 2: Crear query buggy

-- sql/buggy.sql

*-- **BUG**: falta condición de JOIN en order_items*

-- Produce cross join implícito -> totales inflados

```
SELECT c.name,
       SUM(oi.quantity * oi.unit_price) as total_spent
```

```
FROM customers c, orders o, order_items oi
```

```
WHERE o.customer_id = c.id
```

-- Falta: AND oi.order_id = o.id

```
GROUP BY c.name
```

```
ORDER BY total_spent DESC;
```

Paso 3: Pedir a Codex diagnóstico y fix

En Codex TUI

Ejecuta @sql/buggy.sql contra postgres_lab via MCP

usando la tool query.

Explica por qué los resultados son incorrectos.

Identifica el bug con un ejemplo concreto.

Propone sql/fix.sql con la query corregida.

Ejecuta ambas via postgres_lab y compara resultados.

Genera tests/test_queries.py con pytest que:

1. Ejecute fix.sql y verifique que el total de Ana es 1389.98 (no inflado por cross join)
2. Verifique que el número de filas = número de clientes con pedidos (3)

Paso 4: Ejecutar tests

pip install pytest psycopg2-binary --break-system-packages

PYTHONPATH=. pytest tests/test_queries.py -v

Verificación L-PG-2

Verificación	Resultado esperado
sql/buggy.sql	Query con cross join implícito (totales inflados)
Diagnóstico de Codex	Explica el cross join y por qué infla totales
sql/fix.sql	Query con JOIN explícito y resultados correctos
tests/test_queries.py	Tests con invariantes (total Ana = 1389.98, 3 filas)
pytest green	Todos los tests pasan

Limpieza L-PG-2

Igual que L-PG-1: rm -rf + docker compose down -v + limpiar config MCP global si aplica.

7.3 Lab L-PG-3 — Migración incremental controlada

 **Objetivo:** Practicar un migration step sin big-bang: crear cambio DDL pequeño con backward compatibility y rollback verificable.

Paso 1: Crear rol migrator

-- Ejecutar como labadmin (docker compose exec postgres psql -U labadmin -d labdb)

CREATE ROLE codex_migrator **WITH LOGIN PASSWORD** 'migrate2026';

GRANT CONNECT ON DATABASE labdb **TO** codex_migrator;

GRANT USAGE, CREATE ON SCHEMA public TO codex_migrator;

GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO codex_migrator;

GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO codex_migrator;

Paso 2: Conexión para DDL (via psql, no MCP)

Nota: El Reference PostgreSQL MCP Server es read-only por diseño. Para DDL (migraciones), Codex genera los scripts SQL y se ejecutan con psql directamente o via ! en el TUI de Codex.

Verificar acceso del rol migrator

psql "postgres://codex_migrator:migrate2026@localhost:5432/labdb" -c "SELECT 1"

Paso 3: Pedir migración a Codex

En Codex TUI - patrón migration-step del playbook (M12)

Usando el patrón "migration step":

1. Consulta el schema actual de customers via postgres_lab.
2. Genera scripts para añadir columna "phone" (VARCHAR 20, nullable) para backward compatibility.
3. Backfill: pon 'N/A' en los registros existentes.

4. NO hagas NOT NULL todavía (otra migración futura).

5. Genera:

- sql/migrations/001_up.sql (ADD COLUMN + backfill)
- sql/migrations/001_down.sql (DROP COLUMN)
- docs/migration-001.md (plan + riesgo + rollback)

Luego ejecutar manualmente con psql:

```
!psql "postgresql://codex_migrator:migrate2026@localhost:5432/labdb" \
-f sql/migrations/001_up.sql
```

Verificar via MCP (read-only):

Ejecuta `SELECT * FROM customers` via `postgres_lab` y confirma que la columna `phone` existe con valores `'N/A'`.

Rollback:

```
!psql "postgresql://codex_migrator:migrate2026@localhost:5432/labdb" \
-f sql/migrations/001_down.sql
```

Verificación L-PG-3

Verificación	Resultado esperado
sql/migrations/001_up.sql	ALTER TABLE ADD COLUMN phone + UPDATE backfill
sql/migrations/001_down.sql	ALTER TABLE DROP COLUMN phone
docs/migration-001.md	Plan, riesgo, rollback documentados
UP ejecutado	Columna phone presente, valores 'N/A' en existentes
Queries existentes siguen funcionando	Backward compatibility verificada
DOWN ejecutado	Columna phone eliminada, schema original restaurado

Limpieza final (obligatoria — todos los labs)

⚠ **Limpieza completa:** Ejecutar después de cada laboratorio o al finalizar los tres.

```
cd /tmp && rm -rf codex-lab-mcp-pg
docker compose down -v 2>/dev/null
```

Eliminar entrada MCP de `~/.codex/config.toml`

si se añadió de forma global

```
ls /tmp/codex-lab-mcp-pg 2>/dev/null \
&& echo 'ERROR: aún existe' \
|| echo 'OK: limpio'
```

8. Resumen y conceptos clave

Concepto	Detalle
MCP	Estándar abierto para conectar agentes con herramientas externas (host/cliente/servidor).
Transporte STDIO	Proceso local: Codex arranca el servidor MCP y se comunica via stdin/stdout.
Transporte HTTP	Servidor remoto accesible por URL (streamable HTTP).
codex mcp add/list	Comandos CLI para gestionar servidores MCP. Config compartida CLI/IDE.

Concepto	Detalle
config.toml [mcp_servers]	Configuración declarativa: command, args, enabled_tools, startup_timeout_sec.
Reference PostgreSQL MCP	@modelcontextprotocol/server-postgres: read-only, tool query + resources de schema.
enabled_tools / disabled_tools	Control granular de qué tools MCP están disponibles para Codex.
requirements.toml [[mcp_servers]]	Allowlist admin-enforced: id + identity (command o url).
Connection string como arg	La URL de PostgreSQL va en args de config.toml. Wrapper/vault en corporativo.
Usuario read-only	Mínimo privilegio: codex_ro con GRANT SELECT solamente.
Schema-first	Mapear schema antes de generar queries. Reduce alucinaciones.
EXPLAIN via query	Ejecutar EXPLAIN ANALYZE con la tool query del MCP para analizar rendimiento.
Pin de versión MCP	@package@version en vez de latest. Supply chain safety.
Migration step	Patrón incremental: up.sql + down.sql + backward compatibility. DDL via psql.

🎓 **Módulo completado:** Con MCP + PostgreSQL, Codex pasa de generar SQL “a ciegas” a trabajar con el schema real: queries correctas, EXPLAIN, migraciones controladas, y seguridad por defecto.

9. Material de entrega para adopción corporativa

- Checklist de seguridad MCP: allowlist en requirements.toml, pin de versiones, revisión de código del servidor, usuario read-only, wrapper/vault para credenciales.
- Plantilla “DB task”: schema-first, read-only por defecto, verificación obligatoria con tests o conteos.
- Skill opcional db-query-review: SKILL.md que estandariza cómo Codex propone queries, incluye validación y EXPLAIN.
- Config base: .codex/config.toml con postgres_lab configurado, connection string como argumento.
- AGENTS.md con reglas de base de datos: solo SELECT por defecto, guardar queries en .sql, migration-step para DDL via psql.

10. Referencias oficiales

- MCP en Codex: <https://developers.openai.com/codex/mcp/>
- Docs MCP Server: <https://developers.openai.com/resources/docs-mcp/>
- Config Reference (mcp_servers): <https://developers.openai.com/codex/config-reference/>
- Sample config.toml: <https://developers.openai.com/codex/config-sample/>
- Security (MCP allowlist): <https://developers.openai.com/codex/security/>
- CLI Reference (codex mcp): <https://developers.openai.com/codex/cli/reference/>
- Reference PostgreSQL MCP Server: <https://github.com/modelcontextprotocol/servers/tree/main/src/postgres>
- Reference PostgreSQL MCP (npm): <https://www.npmjs.com/package/@modelcontextprotocol/server-postgres>
- Postgres MCP Pro (alternativa): <https://github.com/crystaldba/postgres-mcp>

- Codex Prompting Guide: https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide/